Relational Model

Basic Relational Algebra Operations

- A basic set of relational model operations constitute the relational algebra.

- Mathematical set theory
  - UNION, INTERSECTION, SET DIFFERENCE, and CARTESIAN PRODUCT

- Specific for relational databases.
  - SELECT, PROJECT, JOIN

The SELECT Operation

- The SELECT operation is used to select a subset of the tuples from a relation that satisfy a selection condition.

In general, the SELECT operation is denoted by

$$\sigma_{<\text{select condition}>}(R)$$

The SELECT Operation

- The Boolean expression specified in <selection condition> is made up of clauses of the form

  <attribute name><comparison op><constant value>

  or

  <attribute name><comparison op><attribute name>

The SELECT Operation

- The Boolean conditions AND, OR, and NOT have their normal interpretation as follows:
  - (cond1 AND cond2) is true if both (cond1) and (cond2) are true; otherwise, it is false.
  - (cond1 OR cond2) is true if either (cond1) or (cond2) or both are true; otherwise, it is false.
  - (NOT cond) is true if cond is false; otherwise, it is false.

The SELECT Operation

<attribute name> is the name of an attribute of R,
<comparison op> is normally one of the operators { =, <, ≤ , >, ≥ , ≠ }

Example :

$$\sigma_{(DNO=4\ AND\ SALARY>25000)\ OR\ (DNO=5\ AND\ SALARY>30000)}(EMPLOYEE)$$

The SELECT Operation

Title	Year	Length	Incolor	StudioName	producerC#
Star Wars	1977	124	True	Fox	12345
Mighty Ducks	1991	104	True	Disney	67890
Wayne's world	1992	95	True	Paramount	99999

$\sigma_{length > 100}$ (Movie)

Title	Year	Length	Incolor	StudioName	producerC#
Star Wars	1977	124	True	Fox	12345
Mighty Ducks	1991	104	True	Disney	67890

The PROJECT Operation

- The PROJECT operation, selects certain columns from the table and discards the other columns.

  The resulting form of the PROJECT operation is

  $\Pi_{\text{<attribute list>}}(R)$

The PROJECT Operation

- <attribute list> is a list of attributes from the attributes of relation R.

- The result of the PROJECT operation has only the attributes specified in <attribute list> and in the same order as they appear in the list.

The PROJECT Operation

Title	Year	Length	Incolor	StudioName	producerC#
Star Wars	1977	124	True	Fox	12345
Mighty Ducks	1991	104	True	Disney	67890
Wayne's world	1992	95	True	Paramount	99999

- $\pi_{title,year,length}$ (Movie).

Title	Year	Length
Star Wars	1977	124
Mighty Ducks	1991	104
Wayne's world	1992	95

- $\pi_{incolor}$ (Movie).

Incolor
True

The RENAME Operation

Which can rename either the relation name, or the attribute names or both- in a manner similar to the way we defined SELECT and PROJECT.

- The RENAME operator gives a new schema to a relation.
- R1 := $\text{RENAME}_{R1(A1,\ldots,An)}(R2)$ makes R1 be a relation with attributes A1,…,A*n* and the same tuples as R2.
- Simplified notation: R1(A1,…,A*n*) := R2.

The RENAME Operation

Shop (

name,	addr
Joe's	Maple St.
Sue's	River Rd.

)

R(NAME, ADDR) := Shop

R(

NAME,	ADDR
Joe's	Maple St.
Sue's	River Rd.

)

Set Theoretic Operations

- Union, intersection, and difference.
  - Usual set operations, require both operands have the same relation schema.
  - Two relations must have the same type of tuples; this condition is called union compatibility.

    $R(A_1,A_2,\ldots,A_n)$ and $S(B_1,B_2,\ldots,B_n)$ are said to be union compatible if they have the same degree n, and if $dom(A_i)=dom(B_i)$ for $1 \leq i \leq n$

Set Theoretic Operations

- Set operations on relations
  - R U S: the union of R and S, is the set of elements that are in R or S or both.
  - R ∩ S: the intersection of R and S, is the set of elements that are in both R and S.
  - R – S: the difference of R and S, is the set of elements that are in R but not in S. R – S is different from S – R.

Set Theoretic Operations

Name	Address	Gender	Birthday
Carrie Fisher	123 Maple St., Hollywood	F	9/9/99
Mark Hamill	456 Oak Rd., Brentwood	M	8/8/88

Relation R

Name	Address	Gender	Birthday
Carrie Fisher	123 Maple St., Hollywood	F	9/9/99
Harrison Ford	789 Palm Dr., Beverly Hills	M	7/7/77

Relation S

Set Theoretic Operations

Name	Address	Gender	Birthday
Carrie Fisher	**123 Maple St., Hollywood**	**F**	**9/9/99**
Harrison Ford	**789 Palm Dr., Beverly Hills**	**M**	**7/7/77**
Mark Hamill	**456 Oak Rd., Brentwood**	**M**	**8/8/88**

Relation R U S

Name	Address	Gender	Birthday
Carrie Fisher	**123 Maple St., Hollywood**	**F**	**9/9/99**

Relation R ∩ S

Name	Address	Gender	Birthday
Mark Hamill	**456 Oak Rd., Brentwood**	**M**	**8/8/88**

Relation R – S

Set Theoretic Operations

- Cartesian Product (Product)

  R3 := R1 * R2

  - Pair each tuple t1 of R1 with each tuple t2 of R2.
  - Concatenation t1t2 is a tuple of R3.
  - Schema of R3 is the attributes of R1 and R2, in order.
  - But beware attribute *A* of the same name in R1 and R2: use R1.*A* and R2.*A*.

Set Theoretic Operations

R (

A,	B
1	2
3	4

)

S(

B,	C	D
2	5	6
4	7	8
9	10	11

)

R3(

A,	R.B,	S.B,	C	D
1	2	2	5	6
1	2	4	7	8
1	2	9	10	11
3	4	2	5	6
3	4	4	7	8
3	4	9	10	11

)

Set Theoretic Operations

- Retrieve for each female employee a list of the names of her dependents

FEMALE_EMPS ← $\sigma_{SEX='F'}$ (EMPLOYEE)

EMPNAMES ← $\pi_{LNAME,FNAME,SSN}$ (FEMALE_EMPS)

EMP_DEPENDENTS ← EMPNAMES **X** DEPENDENT

ACTUAL_DEPENDENTS ← $\sigma_{SSN=ESSN}$ (EMP_DEPENDENTS)

RESULT ← $\pi_{LNAME,FNAME,DEPENDENT_NAME}$ (ACTUAL_DEPENDENTS)

The JOIN Operation

- The JOIN operation, denoted by $\rhd\lhd$, is used to combine related tuples from two relations into single tuples.

$$\text{Dept_Mgr} \leftarrow \text{Department} \rhd\lhd_{\text{MGRSSN=SSN}} \text{EMPLOYEE}$$

$$\text{Result} \leftarrow \pi_{\text{DNAME,LNAME, FNAME}} (\text{Dept_Mgr})$$

The JOIN Operation

EMP_DEPENDENTS ← EMPNAMES **X** DEPENDENT

ACTUAL_DEPENDENTS ← $\sigma_{SSN=ESSN}$(EMP_DEPENDENTS)

With a single JOIN operation :

ACTUAL_DEPENDENTS ← EMPNAMES $\triangleright\!\triangleleft_{SSN=ESSN}$ DEPENDENT

The JOIN Operation

- A general JOIN condition is of the form :

<condition> AND <condition> AND ..... AND<condition>

Where each condition is of the form A_i θ B_j , A_i is an attribute of R, B_j is an attribute of S, A_i and B_j have the same domain, and θ (theta) is one of the comparison operators.

The JOIN Operation

EQUIJOIN

- The most common JOIN involves join conditions with equality comparisons only. Such a JOIN, where the only comparison operator used is = , is called an EQUIJOIN.

The JOIN Operation

NATURAL JOIN

- A frequent type of join connects two relations by:
  - Equating attributes of the same name, and
  - Projecting out one copy of each pair of equated attributes.
- Denoted by *

The JOIN Operation

- Example

  First rename the DNUMBER attribute of DEPARTMENT to DNUM – so that it has the same name as the DNUM attribute in PROJECT – then apply NATURAL JOIN :

PROJ_DEPT ← PROJECT *

$\rho_{(DNAME,DNUM,MGRSSN,MGRSTARTDATE)}$ (DEPARTMENT)

DEPT_LOCS ← DEPARTMENT * DEPT_LOCATIONS

The DIVISION Operation

- The DIVISION operation is useful for a special kind of query that sometimes occurs in database applications.

  DIVISION operation, the tuples in the denominator relation restrict the numerator relation by selecting those tuples in the result that match all values present in the denominator.

Example :

“Retrieve the names of employees who work on all the projects that ‘john smith’ works on”

The DIVISION Operation

$$SMITH \leftarrow \sigma_{FNAME='John' \text{ AND } LNAME='Smith'}(EMPLOYEE)$$

$$SMITH_PNOS \leftarrow \pi_{PNO}(WORKS_ON \rhd\lhd_{ESSN=SSN} SMITH)$$

$$SSN_PNOS \leftarrow \pi_{ESSN,PNO}(WORKS_ON)$$

$$SSNS(SSN) \leftarrow SSN_PNOS \div SMITH_PNOS$$

$$RESULT \leftarrow \pi_{FNAME,LNAME}(SSNS * EMPLOYEE)$$

Aggregate Functions and Grouping

- Some of the request that cannot be expressed in the basic relational algebra is to specify mathematical aggregate functions on collections of values from the database.

  Common functions applied to collections of numeric values include SUM, AVERAGE, MAXIMUM, MINIMUM.

  The COUNT function is used for counting tuples or values

Aggregate Functions and Grouping

We can define an AGGREGATE FUNCTION operation, using the symbol ‘F’, to specify these types of requests as follows :

$$\langle \text{grouping attributes} \rangle \mathcal{F}_{\langle \text{function list} \rangle} (R)$$

Where <grouping attributes> is a list of attributes of the relation specified in R, and <function list> is a list of (<function> <attributes>) pairs.

Aggregate Functions and Grouping

R =

A	B
1	3
3	4
3	2

SUM(A)
COUNT(A)
MIN(B)
MAX(B)
AVG(B)

SUM(A) = 1 + 3 + 3 = 7
COUNT(A) = 3
MIN(B) = 2
MAX(B) = 4
AVG(B) = (3 + 4 + 2) / 3 = 3

Aggregate Functions and Grouping

Retrieve each department number, the number of employees in the department, and their average salary.

$$\rho_{R(DNO,NO_OF_EMPLOYEES,AVERAGE_SAL)}({}_{DNO}\mathcal{F}_{COUNT\ SSN,\ AVERAGE\ SALARY}(EMPLOYEE))$$

$${}_{DNO}\mathcal{F}_{COUNT\ SSN,AVERAGE\ SALARY}(EMPLOYEE)$$

$$\mathcal{F}_{COUNT\ SSN,AVERAGE\ SALARY}(EMPLOYEE)$$

OUTER JOIN

- For the natural join operator, it may have dangling tuples (failed to match any tuples in the other relation in the common attributes)

- Outerjoin is formed by starting with R JOIN S, and adding any dangling tuples from R or S, the added tuples must be padded with a special null symbol in all the attributes that they do not possess but that appear in the joint results.

OUTER JOIN

R =

A	B
1	2
4	5

S =

B	C
2	3
6	7

(1,2) joins with (2,3), but the other two tuples are dangling.

R ⟕ S =

A	B	C
1	2	3
4	5	NULL
NULL	6	7

OUTER JOIN

- Left outerjoin, only consider dangling tuples from the relation in the left side.
- Right outerjoin, only consider dangling tuples from the relation in the right side.

Examples

- Retrieve the name and address of all employee who work for the ‘research’ department.

RESEARCH_DEPT ← $\sigma_{DNAME='Research'}$(DEPARTMENT)

RESEARCH_EMPS ← (RESEARCH_DEPT EMPLOYEE)

RESULT ← $\pi_{FNAME,LNAME,ADDRESS}$ (RESEARCH_EMPS)

Examples

- Find the names of employees who work on all the projects controlled by department number 5.

$$DEPT5_PROJS(PNO) \leftarrow \pi_{PNUMBER}(\sigma_{DNUM=5}(PROJECT))$$

$$EMP_PROJ(SSN,PNO) \leftarrow \pi_{ESSN,PNO}(WORKS_ON)$$

$$RESULT_EMP_SSNS \leftarrow EMP_PROJ \div DEPT5_PROJS$$

$$RESULT \leftarrow \pi_{LAME,\ FNAME}\ (RESULT_EMP_SSNS * EMPLOYEE)$$

Examples

- For every project in ‘Stafford’, list the project number, the controlling department number, and the department manager’s last name, address, and birthdate.

STAFFORD_PROJS ← $\sigma_{\text{PLOCATION='Stafford'}}$(PROJECT)

CONTR_DEPT ← (STAFFORD_PROJS $\rhd\lhd_{\text{DNUM=DNUMBER}}$ (DEPARTMENT)

PROJ_DEPT_MGR ← (CONTR_DEPT $\rhd\lhd_{\text{MGRSSN=SSN}}$ EMPLOYEE)

RESULT ← $\pi_{\text{PNUMBER,DNUM,LNAME,ADDRESS,BDATE}}$ (PROJ_DEPT_MGR)

Examples

- List the names of all employees with two or more dependents.

$T1(SSN,NO_OF_DEPTS) \leftarrow {}_{ESSN}\mathcal{F}_{COUNT\ DEPENDENT_NAME}(DEPENDENT)$

$T2 \leftarrow \sigma_{NO_OF_DEPS \geq 2}(T1)$

$RESULT \leftarrow \pi_{LNAME,FNAME}(T2 * EMPLOYEE)$

Examples

- Retrieve the names of employees who have no dependents

ALL_EMPS ← π_{SSN} (EMPLOYEE)

EMPS_WITH_DEPS(SSN) ← π_{ESSN} (DEPENDENT)

EMPS_WITHOUT_DEPS ← (ALL_EMPS - EMPS_WITH_DEPS)

RESULT ← $\pi_{LNAME, FNAME}$ (EMPS_WITHOUT_DEPS * EMPLOYEE)

Examples

- List the names of managers who have at least one dependent.

$MGRS(SSN) \leftarrow \pi_{MGRSSN}(DEPARTMENT)$

$EMPS_WITH_DEPS(SSN) \leftarrow \pi_{ESSN}(DEPENDENT)$

$MGRS_WITH_DEPS \leftarrow (MGRS \cap EMPS_WITH_DEPS)$

$RESULT \leftarrow \pi_{LNAME, FNAME}(MGRS_WITH_DEPS * EMPLOYEE)$

Query Optimization

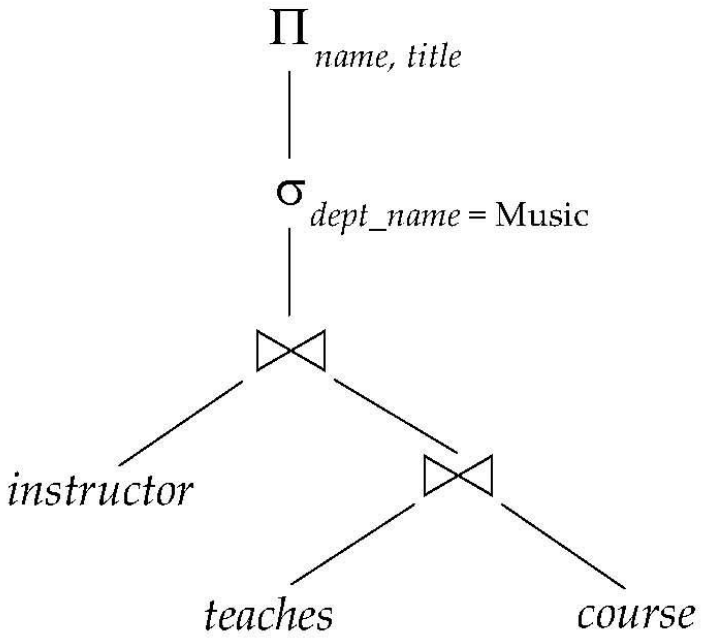
- INSTR(i_id, name, dept_name, …)
- COURSE(c_id, title, …)
- TEACHES(i_id, c_id, …)

- The name of all instructors in the department of Music together with the titles of all courses they teach

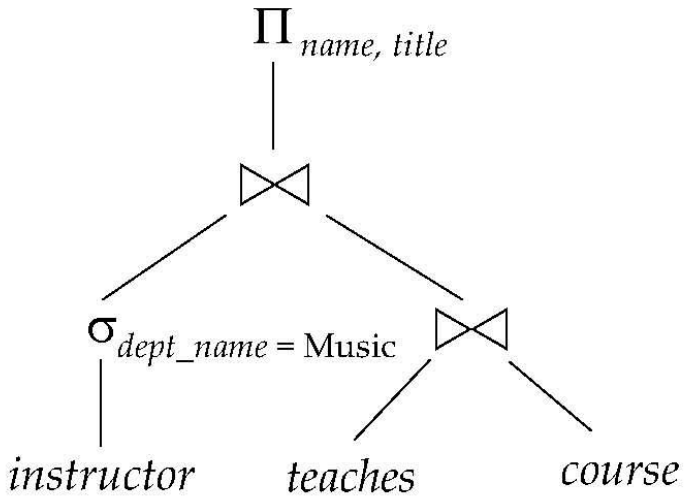
Query Optimization

```
SELECT  I.name, C.title
FROM INSTR I, COURSE C, TEACHES T
WHERE I.i_id = T.i_id
 AND T.c_id = C.c_id
AND dept_name="Music"
```

Query Optimization



$\prod(\sigma(INSTR \quad (TEACHES \quad COURSE)))$



$\prod(\sigma(INSTR) \quad (TEACHES \quad COURSE))$